

O pipeline completo de ML

Aula 5.1 — do dado cru à métrica, com o dataset Iris e o scikit-learn.

O que vamos fazer hoje

01	Objetivos	5 min	02	Recapitulação	10 min	03	Teoria	35 min	04	Quiz rápido	10 min
05	Exercício guiado	45 min	06	Erros comuns	8 min	07	Desafio de casa	5 min	08	Leitura	2 min

Ao final da aula, você consegue

01	02	03
Explicar o que é aprendizado supervisionado	Identificar features, alvo, classes e amostras	Reconhecer a estrutura X e y
04	05	06
Executar o pipeline básico de ponta a ponta	Separar dados de treino e de teste	Treinar um classificador no scikit-learn
07	08	09
Gerar previsões para dados novos	Interpretar accuracy e matriz de confusão	Comparar o modelo com um baseline

Da Aula 3 para hoje

O que você já sabe fazer

DataFrame, indexação, filtro booleano, groupby e gráfico rápido no Matplotlib. Isso carrega, limpa e descreve um dado — e hoje vira a primeira etapa de um ciclo maior.

O que muda hoje

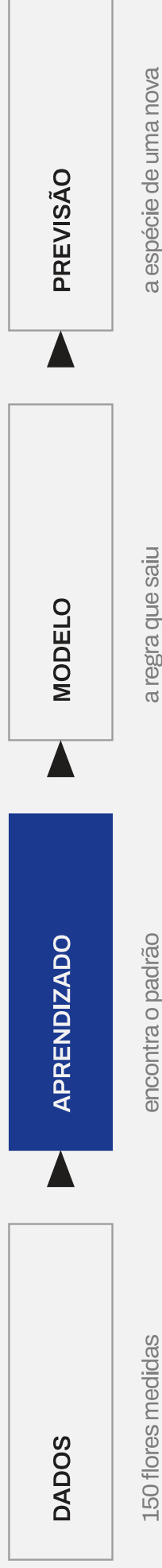
O notebook para de descrever o passado e passa a responder sobre uma flor que nunca viu. Todo o resto da aula é sobre como saber se essa resposta merece confiança.

03

Teoria

O que é machine learning

O modelo encontra padrões nos dados e usa esses padrões para prever.



Aprendizado supervisionado

Cada exemplo vem com a resposta certa. O modelo aprende a relação $X \rightarrow y$.

O que o modelo recebe

Uma tabela de medidas, X , e a coluna de respostas, y , na mesma ordem. Uma linha de X corresponde sempre ao valor de mesma posição em y .

O que ele devolve

Para uma linha nova, um valor de y . Se y é categoria, chamamos de classificação — o caso de hoje. Se é número contínuo, é regressão.

X e y, na tabela do Iris

sepal length	sepal width	petal length	petal width	species
5.1	3.5	1.4	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.3	3.3	6.0	2.5	virginica

X · 150 × 4 · as medidas

y · 150 · o alvo

Pergunta para a turma: se eu quisesse prever a largura da pétala, qual coluna viraria y?

O dataset Iris

150

amostras

uma flor por linha, medida em centímetros

4

características

comprimento e largura de sépala e pétala

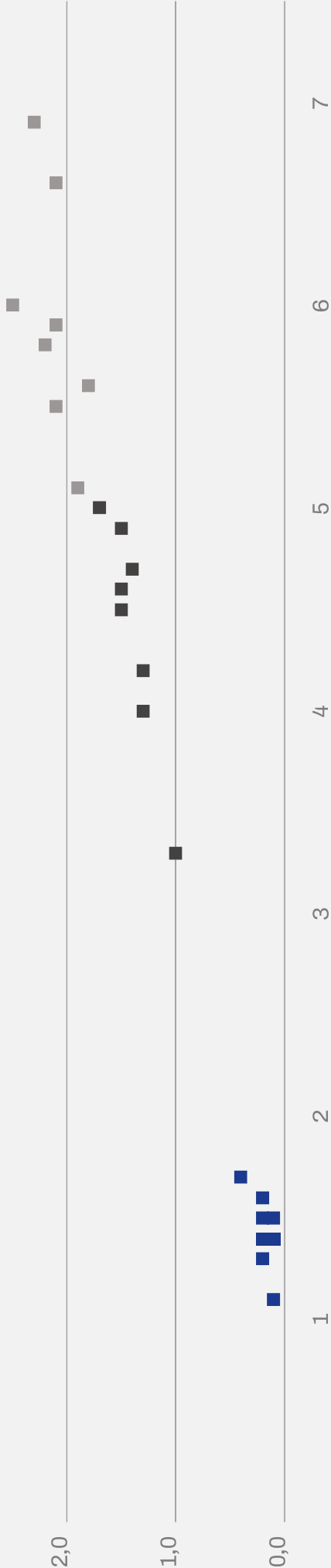
3

classes

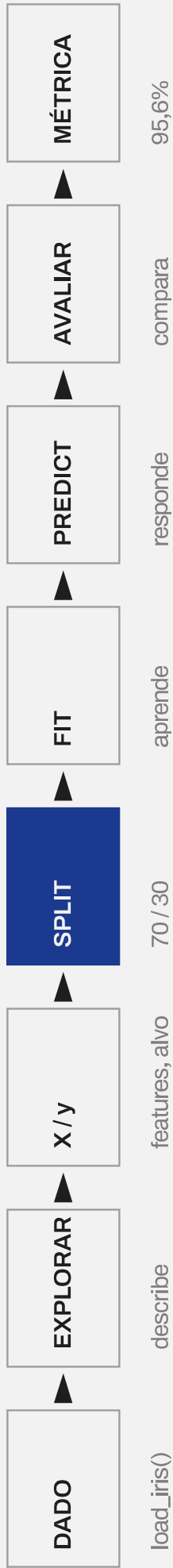
setosa, versicolor e virginica, 50 de cada

As três espécies, em duas medidas

■ setosa ■ versicolor ■ virginica



O pipeline completo



As oito etapas de qualquer projeto supervisionado. A caixa em destaque é a que separa um número honesto de um número bonito.

Treino e teste

150 amostras



o modelo aprende aqui

a prova, aberta uma vez só

Pergunta para a turma: por que não medir o acerto nas mesmas flores que o modelo usou para aprender?

Dois argumentos que não se esquece

`train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)`

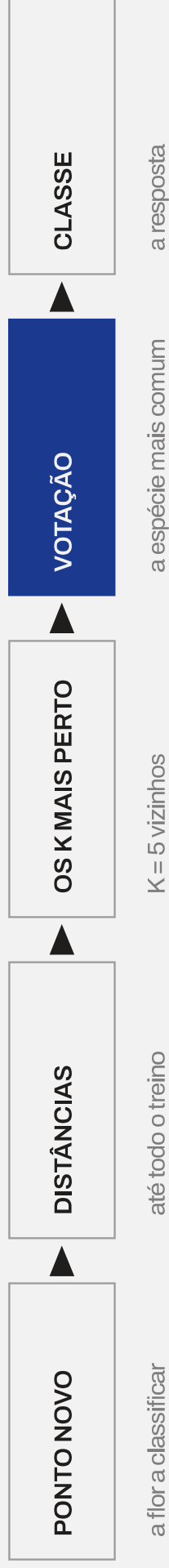
random_state=42

Fixa o sorteio. Rodar duas vezes devolve exatamente o mesmo split — e portanto o mesmo número no fim. Sem ele, cada execução conta uma história diferente.

stratify=y

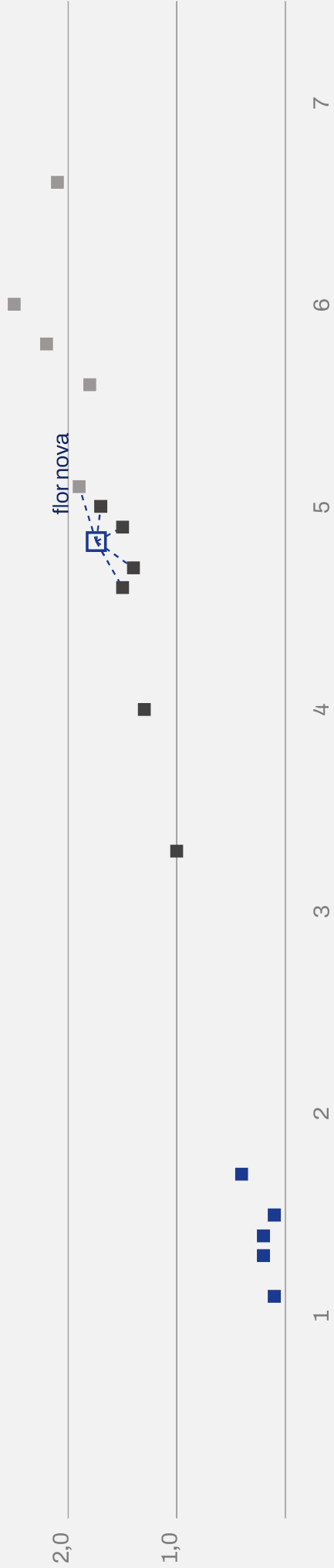
Mantém a proporção das classes nos dois lados: 15 de cada espécie no teste. Sem ele, o sorteio é uniforme sobre as linhas e a proporção varia.

Como o KNN decide



Nada disso acontece no treino; o fit do KNN só guarda a tabela. O trabalho inteiro está na hora de prever.

K = 5, na prática



Três dos cinco vizinhos são versicolor: a previsão é versicolor, com 3 votos contra 2.

Perto quer dizer o quê?

$$d = \sqrt{(\sum (x_i - y_i)^2)}$$

É Pitágoras

A mesma hipotenusa do colégio, só que com quatro medidas em vez de duas.

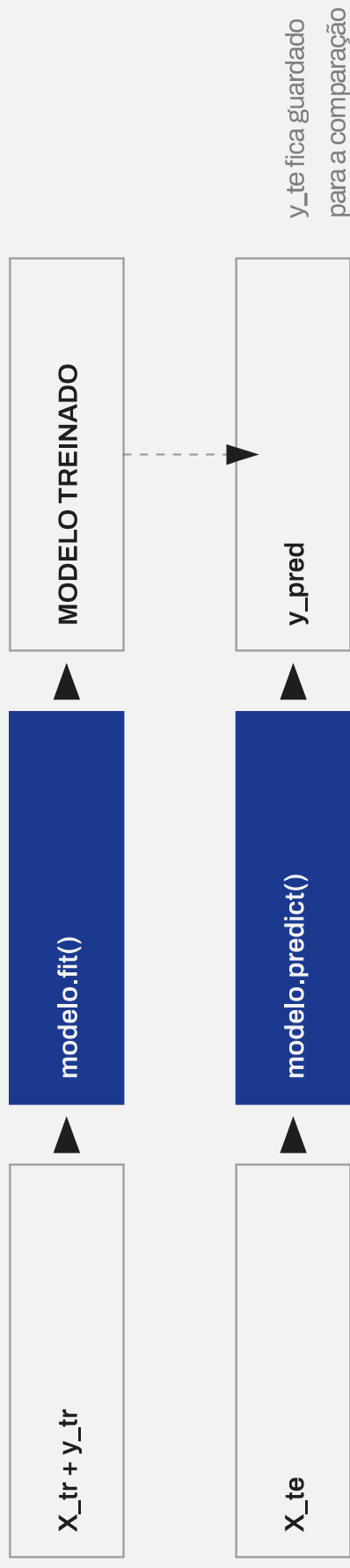
Diferença por diferença

Compara as flores coluna a coluna, eleva ao quadrado e soma tudo.

Escala importa

Colunas em unidades muito diferentes desequilibram a conta. Voltamos a isso na aula 5.4.

fit e predict



Pergunta para a turma: por que o predict não recebe y_te junto?

95,6%

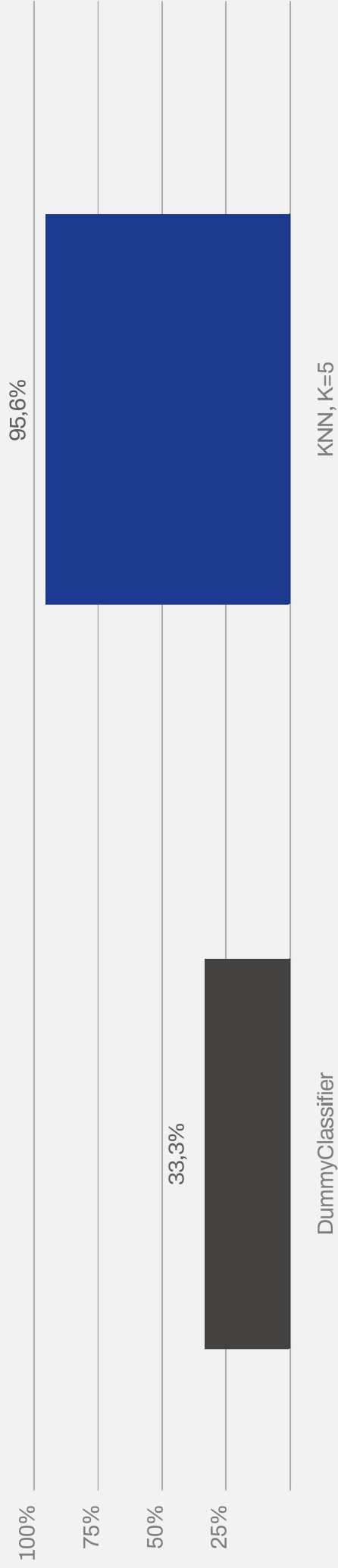
accuracy = previsões corretas / total = 43 acertos em 45 flores do teste.

A matriz de confusão

REAL / PREVISTO	SETOSA	VERSICOLOR	VIRGINICA
setosa	15 ✓	0	0
versicolor	0	14 ✓	1
virginica	0	1	14 ✓

Diagonal é acerto, fora da diagonal é erro. Existem outras métricas — precisão, recall, F1 — e elas entram na aula 6.5.

Comparado com o quê?



Uma métrica só faz sentido quando sabemos com o que comparar.

Dois avisos para as próximas aulas

Escala

O KNN depende de distância, então a escala das colunas influencia o resultado. No Iris tudo está em centímetros e isso não atrapalha. Tratamos escalonamento na aula 5.4.

Treinar bem não é generalizar bem

Acertar muito no treino não garante acertar em dados novos. Vamos estudar isso direito quando introduzirmos validação, na aula 5.2.

04

Quiz rápido

X tem 150 amostras e 4 features. Qual é o shape?

A (4, 150)

B (150, 4)

C (150, 150)

D (4, 4)

Resposta: B

A (4, 150)

B (150, 4)

C (150, 150)

D (4, 4)

Sempre (linhas, colunas). Uma linha por flor, uma coluna por medida.

Mais seis, de boca

01

O que representa y ?

02

Por que separar treino e teste?

03

Para que serve `random_state`?

04

Para que serve `stratify=y`?

05

O que é a diagonal da matriz?

06

Por que comparar com um baseline?

05

Exercício guiado

Carregar e separar X e y

A célula	O resultado
<pre>from sklearn.datasets import load_iris iris = load_iris() X = iris.data y = iris.target print(X.shape, y.shape)</pre>	(150, 4) (150,)

Pergunta: o que significa cada um dos dois números do primeiro shape?

Explorar

A célula

```
import pandas as pd

df = pd.DataFrame(X,
                  columns=iris.feature_names)
df['especie'] = iris.target_names[y]

df.groupby('especie').mean().round(2)
```

Pergunta: qual das quatro medidas parece separar melhor as espécies?

O resultado, em cm

especie	sep	pet	pet
setosa	5.01	3.43	1.46
versicolor	5.94	2.77	4.26
virginica	6.59	2.97	5.55

Separar treino e teste

A célula	O resultado
<pre>from sklearn.model_selection import (train_test_split) X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42) print(X_tr.shape, X_te.shape)</pre>	<pre>(105, 4) (45, 4)</pre>

Pergunta: se eu rodar de novo sem random_state, o que muda?

Criar e treinar o KNN

A célula

```
from sklearn.neighbors import (  
    KNeighborsClassifier)  
  
modelo = KNeighborsClassifier(  
    n_neighbors=5)  
modelo.fit(X_tr, y_tr)
```

O resultado

```
KNeighborsClassifier()
```

Pergunta: o que o modelo guardou quando chamamos fit?

Prever

A célula	O resultado
<pre>y_pred = modelo.predict(X_te) print(y_pred[:8]) print(y_te[:8])</pre>	<pre>[1 0 2 1 1 0 1 2] [1 0 2 1 1 0 1 2]</pre>

Pergunta: as oito primeiras bateram. Isso já quer dizer que o modelo é bom?

Avaliar

A célula	O resultado
<pre>from sklearn.metrics import (accuracy_score, confusion_matrix) print(accuracy_score(y_te, y_pred)) print(confusion_matrix(y_te, y_pred))</pre>	<pre>0.955555555555555556 [[15 0 0] [0 14 1] [0 1 14]]</pre>

Pergunta: onde estão os dois erros, e por que eles não surpreendem?

Comparar com o baseline

A célula	O resultado
<pre>from sklearn.dummy import DummyClassifier base = DummyClassifier(strategy='most_frequent') base.fit(X_tr, y_tr) print(base.score(X_te, y_te))</pre>	0.3333333333333333

Pergunta: nosso 95,6% ficaria impressionante se o baseline desse 94%?

O pipeline inteiro, em uma tela

```
# dado
X, y = load_iris(return_X_y=True)

# split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

# fit e predict
modelo = KNeighborsClassifier(n_neighbors=5).fit(X_tr, y_tr)
y_pred = modelo.predict(X_te)

# avaliar e comparar
print(accuracy_score(y_te, y_pred))
print(confusion_matrix(y_te, y_pred))
print(DummyClassifier(strategy='most_frequent').fit(X_tr, y_tr).score(X_te, y_te))
```

06

Erros comuns

Seis erros para não cometer

01

Prever antes do fit

NotFittedError. O modelo precisa estudar antes da prova.

02

Medir no treino

score(X_tr, y_tr) mede memória. Reporte sempre o teste.

03

Esquecer random_state

Outro número a cada execução, nenhuma conclusão sobrevive.

04

Esquecer stratify

A proporção das classes muda e a comparação fica injusta.

05

Escolher K olhando o teste

Testar dez valores e ficar com o melhor transforma teste em treino.

06

Transformar antes de separar

Informação do teste vaza para o treino. Vamos estudar isso a fundo depois.

Desafio de casa

A tarefa

Refaça as sete etapas com outro dataset de classificação do scikit-learn — `load_wine` ou `load_breast_cancer`. Mesmo modelo, mesmo K , mesma ordem de células.

Checklist: carregar · explorar · X e y · split com stratify e semente · treinar · prever · accuracy · matriz · baseline · explicar.

Entrega

Um notebook com accuracy de teste, matriz de confusão, a comparação com o `DummyClassifier` e três linhas explicando o resultado com as suas palavras.

Leitura

01 `load_iris` [scikit-learn.org · datasets](https://scikit-learn.org/datasets)

02 `train_test_split` [scikit-learn.org · model_selection](https://scikit-learn.org/model_selection)

03 `KNeighborsClassifier` [scikit-learn.org · neighbors](https://scikit-learn.org/neighbors)

04 `accuracy_score` e `confusion_matrix` [scikit-learn.org · metrics](https://scikit-learn.org/metrics)

Aula 5.2

Treino, validação e teste: a disciplina do conjunto trancado — e o que é vazamento de dados.

Desafio corrigido nos primeiros 15 minutos